

Day 40: Insert & Delete in Binary Search Tree (BST)

✓ INSERT in BST

💡 Logic:

Start from root

Compare value:

Smaller → go left

Greater → go right

Insert at the first empty position

🧠 Example:

Insert: 50, 30, 70, 20

```
    50
   /  \
  30   70
 /
20
```

 Java Code (Insert)

```
1. package Day_40;
2.
3. public class BSTinsert {
4.
5.     class Node {
6.         int data;
7.         Node left, right;
8.
9.         Node(int data) {
10.            this.data = data;
11.            left = right = null;
12.        }
13.    }
14.
15.    Node root;
16.
17.    // Insert Function
18.    Node insert(Node root, int key) {
19.        if (root == null) {
20.            return new Node(key);
21.        }
22.
23.        if (key < root.data) {
24.            root.left = insert(root.left, key);
25.        } else if (key > root.data) {
26.            root.right = insert(root.right, key);
27.        }
28.
29.        return root;
30.    }
31.
32.    // ☒ Inorder Traversal
33.    void inorder(Node root) {
34.        if (root != null) {
35.            inorder(root.left);
36.            System.out.print(root.data + " ");
37.            inorder(root.right);
38.        }
39.    }
40.
41.    public static void main(String[] args) {
42.        BSTinsert tree = new BSTinsert();
43.
44.        tree.root = tree.insert(tree.root, 50);
45.        tree.root = tree.insert(tree.root, 30);
46.        tree.root = tree.insert(tree.root, 70);
47.        tree.root = tree.insert(tree.root, 20);
48.        tree.root = tree.insert(tree.root, 40);
49.        tree.root = tree.insert(tree.root, 60);
50.        tree.root = tree.insert(tree.root, 80);
51.
52.        System.out.println("Inorder (Sorted): ");
53.        tree.inorder(tree.root);
54.    }
55. }
56.
```

✖ DELETE in BST (Important ★)

Deletion has 3 cases

◆ Case 1: Node is Leaf (No child)

👉 Simply delete it

Before: After:
20 null

◆ Case 2: Node has 1 Child

👉 Replace node with its child

30 20
/
20

◆ Case 3: Node has 2 Children (MOST IMPORTANT)

👉 Replace with Inorder Successor (smallest in right subtree)

💡 Inorder Successor:

👉 Leftmost node in right subtree

 Java Code (Delete)

```

1. package Day_40;
2.
3. public class BSTdelete {
4.
5.     class Node {
6.         int data;
7.         Node left, right;
8.
9.         Node(int data) {
10.            this.data = data;
11.            left = right = null;
12.        }
13.    }
14.
15.    Node root;
16.
17.    // Insert
18.    Node insert(Node root, int key) {
19.        if (root == null) return new Node(key);
20.
21.        if (key < root.data)
22.            root.left = insert(root.left, key);
23.        else if (key > root.data)
24.            root.right = insert(root.right, key);
25.
26.        return root;
27.    }
28.
29.    // Inorder Traversal (Sorted Output)
30.    void inorder(Node root) {
31.        if (root != null) {
32.            inorder(root.left);
33.            System.out.print(root.data + " ");
34.            inorder(root.right);
35.        }
36.    }
37.
38.    // Find Min (Inorder Successor)
39.    Node minValue(Node root) {
40.        while (root.left != null) {
41.            root = root.left;
42.        }
43.        return root;
44.    }
45.
46.    // Delete Function
47.    Node delete(Node root, int key) {
48.        if (root == null) return root;
49.
50.        if (key < root.data) {
51.            root.left = delete(root.left, key);
52.        }
53.        else if (key > root.data) {
54.            root.right = delete(root.right, key);
55.        }
56.        else {
57.            // Case 1 & 2
58.            if (root.left == null)
59.                return root.right;
60.            else if (root.right == null)
61.                return root.left;
62.
63.            // Case 3: 2 children
64.            Node successor = minValue(root.right);
65.            root.data = successor.data;
66.            root.right = delete(root.right, successor.data);
67.        }

```

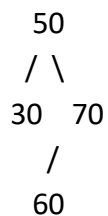
```

68.
69.     return root;
70. }
71.
72. public static void main(String[] args) {
73.     BSTdelete tree = new BSTdelete();
74.
75.     // Insert
76.     tree.root = tree.insert(tree.root, 50);
77.     tree.root = tree.insert(tree.root, 30);
78.     tree.root = tree.insert(tree.root, 70);
79.     tree.root = tree.insert(tree.root, 20);
80.     tree.root = tree.insert(tree.root, 40);
81.     tree.root = tree.insert(tree.root, 60);
82.     tree.root = tree.insert(tree.root, 80);
83.
84.     System.out.print("Before Deletion: ");
85.     tree.inorder(tree.root);
86.
87.     // Delete
88.     tree.root = tree.delete(tree.root, 50);
89.
90.     System.out.print("\nAfter Deletion: ");
91.     tree.inorder(tree.root);
92. }
93. }
94.

```

🔥 Dry Run (Important for Interview)

Delete 50 from:



Step:

Node has 2 children

Inorder successor = 60

Replace 50 → 60

Delete 60 from right subtree

Result:

